

12. L'écran ne s'écrit que pendant le VBlank

La console dessine l'écran ligne par ligne, soixante fois par seconde ; on ne peut lui écrire que pendant ses pauses : le VBlank, et la toute petite pause au bout de chaque ligne.

1. Ce qu'il faut comprendre

L'écran de la Game Boy est dessiné **ligne par ligne**, de haut en bas : 144 lignes, puis une pause de 10 lignes avant de recommencer. Cette pause s'appelle le **VBlank** (« vertical blank »). Elle dure à peine plus d'**une milliseconde**, soixante fois par seconde. Et au bout de **chaque ligne**, il y a aussi une toute petite pause, le temps de revenir à gauche.

Pendant que la console dessine une ligne, la puce graphique **occupe la mémoire vidéo** : le processeur n'a pas le droit d'y écrire. `texte()`, `effacer()` et `nombre()` attendent donc, **avant chaque lettre**, une pause : celle du bout de la ligne, ou le VBlank. Au pire, ils attendent **une ligne** d'écran : un dixième de milliseconde. Une lettre ne coûte presque rien.

Presque rien, mais pas rien. Le programme le **mesure** avec `images()`, l'horloge de la console, qui avance à chaque VBlank. Il remplit **16 lignes** de « # », écran allumé, et recommence **trois fois** : 48 tours, $48 \times 20 = 960$ **lettres**. `i % 16` (le reste, leçon 2) ramène la ligne à 0 après la 15 : 0, 1 ... 15, puis 0, 1 ... 15 encore. Pendant le VBlank, plusieurs lettres passent d'un coup ; entre deux lignes, une ou deux. La mesure donne **3 images** : environ 320 lettres par image. Pour vingt lettres, on ne verrait rien ; pour redessiner tout un décor, ça compte.

Première solution : éteindre l'écran. `ecran(0)` l'éteint ; la mémoire vidéo est alors libre **tout le temps**, et les 960 écritures se font d'un trait, sans attendre. `ecran(1)` le rallume. La mesure donne **1 image** : c'est `ecran(0)` qui l'a prise, car il attend lui-même un VBlank (éteindre l'écran pendant qu'il se dessine peut abîmer une vraie console). L'écran clignote en blanc un instant : on le fait donc aux moments où ça ne gêne pas — au début d'un niveau, entre deux écrans. **À partir d'ici, les leçons dessinent leur décor écran éteint.**

Seconde solution, dans la boucle du jeu : n'écrire que ce qui change. Déplacer un personnage, c'est effacer **une** case et en écrire **une** — pas redessiner toute la ligne. Un jeu qui redessinerait tout l'écran à chaque image passerait une image entière par tour à attendre, et tournerait deux fois moins vite. C'est pourquoi les programmes du chapitre 2 gardaient `ancien`.

Les vrais jeux Game Boy vivent avec cette règle : le décor est posé écran éteint, et chaque image ne change que quelques cases. Ce qui bouge beaucoup est confié aux **lutins**, que le tutoriel présente dans ses leçons sur les sprites.

À toi : remplace le 48 par 16 dans la boucle écran allumé (l'écran rempli une seule fois) : combien d'images, maintenant ? Et avec 1 ?

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **894 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage – la mise en route de la console
main()	202 o	← démarrage – ta fonction
VBlank	8 o	← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage	28 o	← image() – attend l'image suivante
AttendreVBlank	16 o	← ecran() – attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte, EcrireNombre – attend de pouvoir écrire une case
AdresseCase	10 o	← texte() – trouve l'adresse d'une case de l'écran
EcrireTexte	10 o	← texte() – écrit un texte, lettre par lettre
EcrireNombre	33 o	← nombre() – écrit un nombre en chiffres
CopierTuiles	94 o	← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage – vide les deux cartes de l'écran
Tuiles	352 o	← CopierTuiles – 22 lettres sur 43
"#####..."	20 o	← texte() – un texte gravé
"::::::::::::..."	20 o	← texte() – un texte gravé
"ALLUME:"	7 o	← texte() – un texte gravé
"IMAGES"	6 o	← texte() – un texte gravé
"ETEINT:"	7 o	← texte() – un texte gravé
"IMAGES"	6 o	← texte() – un texte gravé

Lettres gravées : AEGILMNSTU0123456789:~

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

texte()	× 6	90 o sur place, dans main() + AdresseCase (10 o), EcrireTexte (10 o)
ecran()	× 2	21 o sur place, dans main() + AttendreVBlank (16 o)
nombre()	× 2	20 o sur place, dans main() + EcrireNombre (33 o)
images()	× 4	8 o sur place, dans main()
image()	× 1	3 o sur place, dans main() + AttendreImage (28 o)

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : mesure combien de temps il faut pour remplir l'écran,
// d'abord écran ALLUMÉ, puis écran ÉTEINT. Éteint, c'est beaucoup plus rapide.
//
// Pourquoi : écran allumé, la console ne peut écrire dans l'image que pendant
// une courte pause entre deux images (le « VBlank »). texte() attend cette pause.
// Écran éteint, rien n'est affiché : on peut écrire à tout moment.

#include <texte>    // écrit un texte à l'écran
#include <ecran>    // éteint ou rallume l'écran
#include <nombre>   // écrit un nombre en chiffres
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() { // Le jeu commence ici.
// Écran allumé : avant chaque lettre, texte() attend une pause.
uint8_t debut = images(); // images() = combien d'images depuis le démarrage.
// On note l'heure de départ.
for (uint8_t i = 0; i < 48; i++) { // 48 tours...
    texte(0, i % 16, "#####"); // 20 lettres ; 3 fois l'écran
// i % 16 = le reste par 16 : 0 à 15, puis encore
// 0 à 15, 3 fois : 3 fois les 16 premières lignes.
}
uint8_t allume = images() - debut; // Heure d'arrivée moins heure de départ :
// le nombre d'images que ça a pris.

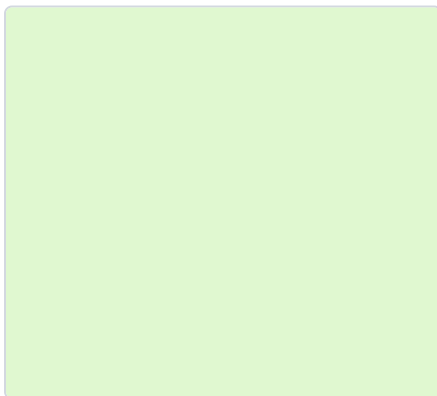
// Écran éteint : la mémoire vidéo est libre.
debut = images(); // Nouvelle heure de départ.
ecran(0); // ecran(0) ÉTEINT l'écran.
for (uint8_t i = 0; i < 48; i++) { // Le même travail...
    texte(0, i % 16, "::::::::::::::::"); // ... avec des « : » pour voir le changement.
}
ecran(1); // ecran(1) le RALLUME.
uint8_t eteint = images() - debut; // Le temps que ça a pris cette fois.

texte(0, 16, "ALLUME:"); nombre(8, 16, allume); // Les deux mesures, en bas.
texte(12, 16, "IMAGES");
texte(0, 17, "ETEINT:"); nombre(8, 17, eteint); // Beaucoup plus petit.
texte(12, 17, "IMAGES");

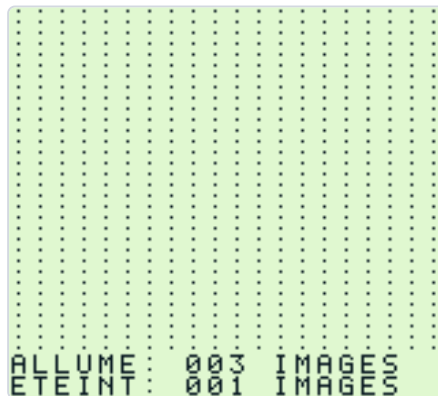
while (true) { // La boucle du jeu.
    image();
}
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Seize lignes de « : » ; en bas, la mesure : 3 images écran allumé pour 960 lettres, 1 écran éteint.

5. Ce que la console répond

Ces contrôles sont ceux que `verification/verifier-cours.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ les seize lignes sont dessinées
- ✓ écran allumé : plusieurs images pour 960 lettres (3 images)
- ✓ écran éteint : presque rien (1)
- ✓ l'écran affiche la mesure

← [11. for : la boucle qui compte](#)

[sommaire](#)

[13. Compter à l'envers, et le piège du zéro](#) →